

Java Exception Handling

Part 2: Checked vs Unchecked (Deep Dive), try-with-resources, Custom Exceptions, Exception Chaining & Best Practices

This document picks up where the introductory notes left off. It assumes you already know the exception hierarchy, basic try/catch/finally mechanics, and the throw/throws keywords. The goal here is to go deep on the topics that were only briefly mentioned or missing entirely: why checked vs unchecked exceptions exist as a design choice, how try-with-resources solves a real resource-leak problem, how to design your own exception classes properly, how exceptions chain to preserve root causes, and the practices that separate clean exception handling from messy code.

Contents

1. Checked vs Unchecked Exceptions — The Design Philosophy
2. try-with-resources
3. Custom Exceptions
4. Exception Chaining
5. Best Practices

1. Checked vs Unchecked Exceptions — The Design Philosophy

You already know the mechanical difference: checked exceptions are verified by the compiler at compile-time, unchecked exceptions are not. What matters more is understanding **why** Java splits exceptions this way, because that reasoning is what tells you which kind to use when you design your own exceptions later in this document.

1.1 The Core Idea: Recoverable vs Programming Error

Java's exception hierarchy encodes a judgment call about every abnormal condition: *can the calling code reasonably be expected to recover from this, or does it indicate a bug that needs to be fixed?*

- **Checked exceptions** represent conditions external to the program that a well-written application should anticipate and recover from — a file that doesn't exist, a network connection that drops, a database that's unreachable. These are not bugs; they are expected failure modes of the real world. The compiler forces you to acknowledge them because forgetting to handle a missing file is a real risk.
- **Unchecked exceptions** (`RuntimeException` and its subclasses) usually represent programming errors — dereferencing a null reference, dividing by zero, indexing an array out of bounds, casting to the wrong type. These conditions are almost always avoidable by writing correct code (a null check, a bounds check). Since they can occur anywhere and forcing every method to declare them would clutter every signature in the codebase, the compiler does not require you to catch or declare them.
- **Errors** (like `OutOfMemoryError` or `StackOverflowError`) represent conditions so severe that the application usually cannot do anything meaningful in response — they signal a broken JVM state rather than a broken program.

1.2 Why the Compiler Cares Only About Checked Exceptions

If every exception were checked, method signatures would balloon: a single call chain touching ten methods might need to declare fifteen exception types, most of which represent bugs rather than recoverable conditions. Java's designers decided that only exceptions representing genuinely recoverable, externally-caused failures deserve compile-time enforcement. This is sometimes called the **"handle or declare"** rule: any code that calls a method declaring a checked exception must either catch it in a try-catch block or re-declare it with its own throws clause, propagating the responsibility up the call stack until something does handle it.

1.3 The Complete Picture

Aspect	Checked Exception	Unchecked Exception	Error
Extends	Exception (not <code>RuntimeException</code>)	<code>RuntimeException</code>	Error
Compiler enforcement	Must catch or declare (throws)	No enforcement	No enforcement
Represents	External/recoverable failure	Programming bug	Fatal JVM condition

Typical examples	IOException, SQLException	NullPointerException, ArithmeticException	OutOfMemoryError, StackOverflowError
Should you catch it?	Yes, when recovery is possible	Usually fix the code instead	Rarely, if ever

1.4 A Common Misconception

A frequent mistake is assuming "checked = serious" and "unchecked = minor." That's not the distinction. A `NullPointerException` can crash a production system just as badly as an unhandled `IOException`. The real distinction is **recoverability and expectedness**: checked exceptions are conditions you plan for as part of normal program design (e.g. "what if the file isn't there?"); unchecked exceptions are conditions you prevent through correct code (e.g. always checking for null before calling a method on an object).

1.5 The Controversy

It's worth knowing that checked exceptions are a point of ongoing debate in the Java community. Critics argue they lead to "exception swallowing" (developers writing empty catch blocks just to satisfy the compiler) and awkward API design. This is why later JVM languages like Kotlin and Scala dropped checked exceptions entirely, and even within Java, the Stream API and functional interfaces (introduced in Java 8) generally avoid checked exceptions because they don't compose well with lambdas. Understanding this context helps explain some of the best practices in Section 5.

2. try-with-resources

2.1 The Problem It Solves

Before Java 7, closing a resource (a file stream, a database connection, a socket) safely required explicit cleanup in a finally block. This is verbose and, more importantly, easy to get subtly wrong.

```
// The old, error-prone way
FileInputStream fis = null;
try {
    fis = new FileInputStream("data.txt");
    // read from fis
} catch (IOException e) {
    System.out.println(e);
} finally {
    if (fis != null) {
        try {
            fis.close();           // closing can ALSO throw IOException
        } catch (IOException e) {
            System.out.println(e);
        }
    }
}
```

Notice the nested try-catch just to close the file safely. If you forget the null check, or forget to catch the exception that `close()` itself can throw, you either get a `NullPointerException` or leak the resource. Multiply this by every file, socket, or connection your program opens, and the boilerplate (and risk of mistakes) adds up quickly.

2.2 The Solution: try-with-resources

Introduced in Java 7, try-with-resources lets you declare a resource directly in the try statement. Java then guarantees the resource's `close()` method is called automatically at the end of the block — whether the block completes normally or exits due to an exception. You no longer need a finally block just for cleanup.

```
// The modern way
try (FileInputStream fis = new FileInputStream("data.txt")) {
    // read from fis
} catch (IOException e) {
    System.out.println(e);
}
// fis.close() is called automatically here, even if an exception occurred
```

2.3 The AutoCloseable Contract

Any object used inside the parentheses of a try-with-resources statement must implement the **java.lang.AutoCloseable** interface (`java.io.Closeable`, used by I/O classes, extends it). This interface has exactly one method:

```
public interface AutoCloseable {
    void close() throws Exception;
}
```

Most standard I/O classes (`FileInputStream`, `FileReader`, `BufferedReader`, `Scanner`, `Connection` in JDBC, etc.) already implement this. If you write your own resource class (say, a custom connection wrapper), you make it eligible for try-with-resources simply by implementing `AutoCloseable` and writing a `close()` method that releases the resource.

```
class MyResource implements AutoCloseable {
    MyResource() {
        System.out.println("Resource opened");
    }
    void use() {
        System.out.println("Using resource");
    }
    @Override
    public void close() {
        System.out.println("Resource closed automatically");
    }
}

public class Demo {
    public static void main(String[] args) {
        try (MyResource r = new MyResource()) {
            r.use();
        }
    }
}
```

```
}  
// Output:  
// Resource opened  
// Using resource  
// Resource closed automatically
```

2.4 Multiple Resources

You can declare more than one resource in the same try statement, separated by semicolons. They are closed in the **reverse order** of their declaration (last opened, first closed) — the same pattern you'd expect from nested finally blocks.

```
try (FileInputStream in = new FileInputStream("in.txt");  
    FileOutputStream out = new FileOutputStream("out.txt")) {  
    // use both in and out  
} catch (IOException e) {  
    System.out.println(e);  
}  
// out is closed first, then in
```

2.5 Suppressed Exceptions

A subtle but important behavior: if an exception is thrown both inside the try block AND from the automatic close() call, the exception from the try block is the one that propagates — the exception from close() is not lost, it's attached as a **suppressed exception**, retrievable via getSuppressed() on the main exception. This is strictly better than the old finally-block approach, where an exception thrown during cleanup would silently overwrite (and hide) the original exception from the try block.

2.6 Java 9+ Enhancement: Effectively Final Resources

Since Java 9, you don't need to declare a new variable inside the try statement — you can use an existing effectively-final variable directly:

```
BufferedReader reader = new BufferedReader(new FileReader("data.txt"));  
try (reader) {           // Java 9+: reuse an existing variable  
    System.out.println(reader.readLine());  
}
```

Key takeaway: Prefer try-with-resources over manual finally-block cleanup for anything that implements AutoCloseable. It is shorter, and it eliminates an entire class of resource-leak and exception-swallowing bugs.

3. Custom Exceptions

3.1 Why Create Your Own Exception Classes

Java's built-in exceptions (IOException, ArithmeticException, IllegalArgumentException, etc.) are generic. When you're building a real application — say, a banking system or an e-commerce checkout — generic

exceptions don't communicate business-specific meaning. Compare these two:

```
throw new Exception("not enough balance");           // generic, vague
throw new InsufficientFundsException(balance, amount); // specific, self-documenting
```

Custom exceptions let you: (1) give errors meaningful, domain-specific names; (2) attach extra data relevant to that failure (e.g. the account balance and requested amount); and (3) let calling code catch precisely the failure it knows how to handle, rather than a broad, generic type.

3.2 Extending Exception vs RuntimeException

The single most important design decision for a custom exception is which class to extend, because that decision determines whether it becomes checked or unchecked:

- Extend **Exception** to create a **checked** custom exception — use this when callers should be forced to acknowledge and handle the failure (e.g. a validation failure that the caller must recover from).
- Extend **RuntimeException** to create an **unchecked** custom exception — use this when the failure represents a violated precondition or programming error that callers generally shouldn't be forced to catch everywhere (e.g. an internal invariant being broken).

3.3 Writing a Checked Custom Exception

```
// Custom checked exception
class InsufficientFundsException extends Exception {
    private double shortfall;

    public InsufficientFundsException(String message, double shortfall) {
        super(message);           // pass message to Exception's constructor
        this.shortfall = shortfall;
    }

    public double getShortfall() {
        return shortfall;
    }
}

class BankAccount {
    private double balance;

    BankAccount(double balance) {
        this.balance = balance;
    }

    void withdraw(double amount) throws InsufficientFundsException {
        if (amount > balance) {
            throw new InsufficientFundsException(
                "Cannot withdraw " + amount + ", balance is " + balance,
                amount - balance
            );
        }
        balance -= amount;
    }
}
```

```

public class Demo {
    public static void main(String[] args) {
        BankAccount acc = new BankAccount(500);
        try {
            acc.withdraw(700);
        } catch (InsufficientFundsException e) {
            System.out.println(e.getMessage());
            System.out.println("Short by: " + e.getShortfall());
        }
    }
}
// Output:
// Cannot withdraw 700.0, balance is 500.0
// Short by: 200.0

```

3.4 Writing an Unchecked Custom Exception

```

// Custom unchecked exception
class InvalidOrderStateException extends RuntimeException {
    public InvalidOrderStateException(String message) {
        super(message);
    }
}

class Order {
    private String status = "PLACED";

    void ship() {
        if (!status.equals("PLACED")) {
            throw new InvalidOrderStateException(
                "Cannot ship an order in state: " + status
            );
        }
        status = "SHIPPED";
    }
}

```

Notice callers of `ship()` are NOT forced to declare or catch `InvalidOrderStateException` — it behaves like `NullPointerException`, surfacing only when something is genuinely wrong with program logic.

3.5 Best-Practice Constructors

A well-designed custom exception typically provides four constructors, mirroring the ones `Exception` itself provides, so it plays nicely with exception chaining (Section 4):

```

class AppException extends Exception {
    public AppException() {
        super();
    }
    public AppException(String message) {
        super(message);
    }
    public AppException(String message, Throwable cause) {

```

```

        super(message, cause);
    }
    public AppException(Throwable cause) {
        super(cause);
    }
}

```

3.6 Naming Convention

By convention, custom exception class names end with **Exception** (e.g. `UserNotFoundException`, `InvalidInputException`). This isn't enforced by the compiler, but every experienced Java developer expects it, and breaking the convention makes code harder to read.

4. Exception Chaining

4.1 The Problem: Losing the Original Cause

It's common to catch a low-level exception and wrap it in a higher-level, more meaningful one before rethrowing — for example, catching a raw `SQLException` inside a data-access method and rethrowing it as a custom `DataAccessException` that the rest of the application understands. But if you're not careful, you can accidentally discard the original exception, losing the real root cause:

```

// BAD: original exception is lost
try {
    connectToDatabase();
} catch (SQLException e) {
    throw new DataAccessException("Could not access data"); // no reference to 'e'!
}

```

Whoever catches `DataAccessException` later, or reads the stack trace in a log file, has no idea the real problem was a `SQLException` — the trail goes cold at the point of the rethrow.

4.2 The Solution: Chaining

Exception chaining (also called "wrapping") means passing the original exception as the **cause** of the new exception, so the full chain of causation is preserved. Since Java 1.4, `Throwable` has built-in support for this via a constructor that accepts a cause, and via the `initCause()` method.

```

// GOOD: original exception preserved as the cause
try {
    connectToDatabase();
} catch (SQLException e) {
    throw new DataAccessException("Could not access data", e); // e becomes the cause
}

```

This works because `Throwable` (the superclass of both `Exception` and `Error`) defines constructors that accept a cause:


```
public Throwable(String message, Throwable cause)
public Throwable(Throwable cause)

public Throwable getCause()    // retrieves the original exception later
```

4.3 What the Stack Trace Looks Like

When a chained exception is printed (e.g. via `printStackTrace()`, or when it propagates uncaught), Java prints the full chain, clearly separated by "Caused by:", so nothing is hidden:

```
Exception in thread "main" DataAccessException: Could not access data
    at Demo.main(Demo.java:14)
Caused by: java.sql.SQLException: Connection refused
    at Demo.connectToDatabase(Demo.java:9)
    ... 1 more
```

4.4 Full Working Example

```
class DataAccessException extends RuntimeException {
    public DataAccessException(String message, Throwable cause) {
        super(message, cause);
    }
}

public class Demo {
    static void connectToDatabase() throws Exception {
        throw new java.sql.SQLException("Connection refused");
    }

    public static void main(String[] args) {
        try {
            connectToDatabase();
        } catch (Exception e) {
            throw new DataAccessException("Could not access data", e);
        }
    }
}
```

Later, anyone catching `DataAccessException` can still dig into the root cause programmatically:

```
try {
    doSomething();
} catch (DataAccessException e) {
    System.out.println("High-level message: " + e.getMessage());
    Throwable rootCause = e.getCause();
    System.out.println("Root cause: " + rootCause);
}
```

4.5 Chaining vs Suppressed Exceptions

Don't confuse exception **chaining** (Section 4, an explicit cause-and-effect relationship you set up deliberately when wrapping one exception in another) with **suppressed exceptions** (Section 2.5, an automatic mechanism used by try-with-resources when a `close()` failure happens alongside a try-block failure). Both preserve information that would otherwise be lost, but they solve different problems and are

surfaced differently (`getCause()` vs `getSuppressed()`).

5. Best Practices

These are the practical rules experienced Java developers follow. Each one is a direct consequence of the mechanics covered above.

5.1 Never Swallow Exceptions Silently

```
// BAD – the failure disappears without a trace
try {
    riskyOperation();
} catch (Exception e) {
    // empty – do NOT do this
}
```

An empty catch block hides bugs and makes production issues nearly impossible to diagnose. At minimum, log the exception (including its stack trace); ideally, handle it meaningfully or rethrow it.

5.2 Catch Specific Exceptions, Not Generic Ones

```
// BAD – hides which exception actually occurred, and catches things you didn't expect
try {
    parseAndSave(input);
} catch (Exception e) { ... }

// GOOD – handles exactly the failures you anticipated
try {
    parseAndSave(input);
} catch (NumberFormatException e) {
    // handle bad input format
} catch (IOException e) {
    // handle save failure
}
```

Catching `Exception` or `Throwable` broadly can accidentally swallow unrelated bugs (like a `NullPointerException` from a typo) alongside the failure you meant to handle.

5.3 Never Catch `Throwable` or `Error`

Errors like `OutOfMemoryError` or `StackOverflowError` indicate the JVM itself is in trouble. Catching them rarely helps and can mask a situation where the application should simply be allowed to fail fast.

5.4 Use `Finally` (or `try-with-resources`) for Cleanup, Always

Any resource you open (file, socket, connection, lock) must be released regardless of whether an exception occurs. Prefer `try-with-resources` (Section 2); fall back to `finally` only for non-resource cleanup.

5.5 Don't Use Exceptions for Normal Control Flow

```
// BAD – using an exception to check for existence is slow and unclear
try {
    return map.get(key);
} catch (NullPointerException e) {
    return defaultValue;
}

// GOOD – use the language feature meant for this
return map.getOrDefault(key, defaultValue);
```

Exceptions are for exceptional, abnormal conditions — not for routine branching logic. Constructing and throwing an exception is significantly more expensive than a normal conditional check, since it captures a full stack trace.

5.6 Always Chain the Original Cause When Wrapping

As shown in Section 4, whenever you catch one exception and throw a different, higher-level one, pass the original as the cause. This is one of the most commonly skipped practices and one of the most damaging to debuggability when skipped.

5.7 Include Useful, Specific Messages

```
// BAD
throw new IllegalArgumentException("bad input");

// GOOD – tells you exactly what was wrong and what value caused it
throw new IllegalArgumentException("Age must be non-negative, got: " + age);
```

5.8 Document Checked Exceptions with Javadoc

```
/**
 * Withdraws the given amount from the account.
 *
 * @param amount the amount to withdraw
 * @throws InsufficientFundsException if the account balance is
 *         lower than the requested amount
 */
void withdraw(double amount) throws InsufficientFundsException { ... }
```

Since checked exceptions are part of a method's contract, document why and when they're thrown so callers know how to react.

5.9 Prefer Unchecked Exceptions for Library/API Design (Modern Convention)

Many modern frameworks (Spring, Hibernate) deliberately favor unchecked exceptions even for what feels like a recoverable failure, because checked exceptions don't compose well through layers of abstraction and lambda-based code. This is a matter of ongoing debate (see Section 1.5) — but it's worth knowing that "use checked exceptions for everything recoverable" is not a universally followed rule in modern Java codebases.

5.10 Clean Up in Reverse Order of Acquisition

When manually managing multiple resources (or relying on try-with-resources, which does this automatically — see Section 2.4), always release them in the reverse order you acquired them, mirroring how nested scopes naturally unwind.

5.11 Fail Fast

Validate inputs and preconditions as early as possible (often by throwing unchecked exceptions like `IllegalArgumentException` or `NullPointerException` immediately) rather than letting invalid state propagate deep into the program where the eventual failure is harder to trace back to its source.

Summary: Use checked exceptions for recoverable, expected failures and unchecked exceptions for programming errors; prefer try-with-resources for anything implementing `AutoCloseable`; design custom exceptions with meaningful names and full constructor sets; always chain the original cause when wrapping an exception; and never let a catch block silently swallow a failure.